

# Investigación comparativa — Traffic Intersection Optimizer

Auditoría técnica del código, contraste con el alcance original y benchmark contra software profesional de ingeniería de tránsito.

Fecha: 2026-05-21 · Versión del proyecto auditada: 0.1.0 · Commit inicial `c66cfc2`

## 1. Resumen ejecutivo

Este documento responde a una pregunta concreta: **¿qué estamos haciendo bien y qué podemos hacer mejor?** No es un resumen de marketing; es una auditoría línea por línea del código realmente entregado, contrastada con (a) el alcance que prometió el asistente que originó el proyecto y (b) el software estándar de la industria.

**Veredicto en una frase:** el proyecto es un *motor de cálculo correcto y honesto para análisis preliminar*, con bases teóricas reconocibles y bien transcritas, pero con **simplificaciones que hoy no están todas declaradas al usuario, un estándar de referencia dos ediciones atrasado y cero pruebas automatizadas**. Es sólido como herramienta de tamizaje y docencia; no sustituye a Synchro, Vissim ni SUMO para un estudio de ingeniería formal — y el código nunca debería dar a entender que sí.

Los cinco hallazgos de mayor severidad:

| #   | HALLAZGO                                                                                | SEVERIDAD |
|-----|-----------------------------------------------------------------------------------------|-----------|
| F22 | No existe ni una sola prueba automatizada para un motor de cálculo numérico             | Crítico   |
| F11 | Las llegadas de la simulación <b>no son Poisson</b> pese a que el código lo afirma      | Alto      |
| F6  | La "cola 95-percentil" es <code>2 × cola_promedio</code> , no el modelo de cola del HCM | Alto      |
| F10 | Se cita HCM 2010 (5ª ed.); el estándar vigente es HCM 7ª ed. (2022) + 7.1 (2025)        | Alto      |
| F1  | Webster (1958) sobrestima el ciclo justo en el régimen congestionado del caso real      | Alto      |

Ninguno es un defecto fatal. Todos son corregibles y la mayoría con esfuerzo moderado. El propósito de este informe es priorizarlos.

## 2. Metodología de la investigación

- Auditoría de código.** Se leyeron íntegros los ocho módulos del backend (`models.py`, `optimizer.py`, `analysis.py`, `simulator.py`, `scenarios.py`, `unsignalized.py`, `data.py`, `main.py`). Cada hallazgo cita archivo y línea para que sea verificable y reproducible.
- Contraste de alcance.** Se comparó el código contra los siete módulos que el asistente original (GitHub Copilot) prometió al cerrar su plan.
- Verificación de estándares.** Se consultaron fuentes primarias sobre la edición vigente del *Highway Capacity Manual* y sobre las limitaciones documentadas del método Webster.
- Benchmark de industria.** Se contrastó el alcance funcional contra HCS, Synchro/SimTraffic, PTV Vissim/Vistro, SUMO y Aimsun, con datos de capacidad verificados (ver §5 y Fuentes).

Lo que **no** cubre esta investigación: no se ejecutó una validación numérica cruzada contra HCS/Synchro con un caso idéntico (no se dispone de licencias); las afirmaciones de fidelidad se basan en la lectura de las fórmulas, no en una comparación de resultados. Esa validación cruzada es, de hecho, una de las recomendaciones (ver §8).

### 3. Auditoría técnica del código

Para cada módulo: qué implementa, qué tan fiel es al estándar que cita, y los hallazgos concretos.

#### 3.1 Optimización de ciclo — `optimizer.py`

**Qué hace.** Implementa el método de Webster (1958): calcula el flujo crítico por fase  $y_i = \max(v/s)$ , su suma  $Y$ , el tiempo perdido  $L$ , y el ciclo óptimo  $Co = (1.5 \cdot L + 5) / (1 - Y)$ . Reparte el verde proporcional a  $y_i/Y$ . Acota el ciclo a  $[40, 120]$  s y, si  $Y \geq 0.95$ , declara sobre-saturación y aplica ciclo máximo.

**Fidelidad.** La transcripción de la fórmula de Webster es correcta. El manejo de sobre-saturación y el acotamiento del ciclo son decisiones de ingeniería razonables y explícitas.

#### Hallazgos:

- **F1 [Alto] — Webster sobrestima el ciclo en congestión.** La literatura es consistente: la fórmula de Webster *sobrestima el ciclo óptimo y produce ciclos irrealmente largos cuando el grado de saturation supera ~0.5*. El caso de ejemplo del propio proyecto opera en LOS E/F con  $x$  cercano o superior a 1 — es decir, **exactamente el régimen donde Webster es menos confiable**. El tope de 120 s (`optimizer.py:60`) y el conmutador  $Y \geq 0.95$  (`optimizer.py:51`) mitigan el extremo, pero **la banda  $0.5 \leq Y < 0.95$  queda sin protección**: ahí Webster puede recomendar ciclos largos que aumentan la demora en lugar de reducirla.
- **F2 [Medio] — Bug de re-acotamiento del verde.** Los verdes se acotan a `[min_green, max_green]` (`optimizer.py:74`) y **después** se reescalán para que la suma cuadre con el ciclo (`optimizer.py:87-92`). Tras el reescalado no se vuelve a acotar: un verde puede terminar por debajo de `min_green` o por encima de `max_green`. El plan resultante puede ser inviable.
- **F4 [Medio] — Un solo algoritmo.** El plan de Copilot prometía "algoritmos" (plural). Solo hay Webster. No hay minimización directa de demora HCM, ni búsqueda heurística, ni optimización por tipo de fase (lead/lag).
- **F5 [Medio] — Verde "efectivo" vs. verde "visualizado".** El optimizador reparte verde *efectivo* (`cycle - L`), pero el planificador de la simulación trata `phase_green` como verde *visualizado* y le suma amarillo y todo-rojo por separado (`simulator.py:48-53`). La distinción efectivo/visualizado se diluye; en rigor difieren en el arranque perdido y la extensión.
- **F3 [Bajo] — Tiempo perdido con aritmética muerta.** En `models.py:81` el tiempo perdido por fase es `2.0 + yellow + all_red - 2.0`, que se reduce a `yellow + all_red`. El `+2 - 2` no hace nada; la intención (separar arranque perdido de despeje) no queda codificada.

#### 3.2 Análisis de capacidad y demora — `analysis.py`

**Qué hace.** Modelo de demora del HCM 2010, cap. 18: capacidad  $c = s \cdot g/C$ , grado de saturación  $x = v/c$ , demora uniforme  $d_1$ , demora incremental  $d_2$ , demora total  $d = d_1 \cdot PF + d_2$ , y LOS A-F por umbrales de demora.

**Fidelidad.** Las fórmulas  $d_1$  y  $d_2$  están **correctamente transcritas** del HCM. El uso de  $\min(1, x)$  en el denominador de  $d_1$  y el capado de  $x$  para estabilidad numérica son correctos. Los umbrales LOS (10/20/35/55/80 s) son correctos y, de hecho, no han cambiado entre ediciones del HCM.

#### Hallazgos:

- **F6 [Alto] — La "cola 95-percentil" no es la del HCM.** En `analysis.py:100-101` la cola es `q_avg = (v/3600) * r * factor` y luego `q_95 = 2 * q_avg`. El factor 2 es una heurística; **no** es el modelo de cola del HCM 2010, que descompone la cola de fin de fase en un término uniforme y uno incremental/aleatorio con un factor de percentil. El campo se llama `queue_95th_veh` y el informe lo presenta como "cola 95-percentil": **se etiqueta como una métrica normativa algo que es una aproximación gruesa.**
- **F8 [Alto] — Flujo de saturación casi sin ajustes.** El HCM parte de 1900 veh/h/carril y aplica ~11 factores (ancho de carril, pendiente, vehículos pesados, estacionamiento, bloqueo de buses, tipo de área, utilización de carriles, radio de giro, peatones/ciclistas). El modelo solo aplica un factor de 0.85 si el carril es compartido (`models.py:44`) y traslada los vehículos pesados al lado de la demanda vía `pcu_factor`. La capacidad calculada puede desviarse de forma material de la real.
- **F10 [Alto] — Estándar de referencia atrasado.** Se cita HCM 2010 (5ª edición). La edición vigente es la **7ª (HCM 2022)**, con la **actualización 7.1 de noviembre de 2025**. Entre la 5ª y la 7ª se recalibraron coeficientes (notablemente glorietas, ver F17), se ampliaron los modos peatonal/ciclista y la 7ª añadió análisis de vehículos conectados y automatizados (CAV). El núcleo `d = d1 * PF + d2` sobrevive entre ediciones, así que esto **no invalida** el cálculo, pero sí lo deja desactualizado para un estudio formal.
- **F7 [Bajo] — Factor de progresión fijo.** `PF = 1.0` constante (`analysis.py:31`). Es coherente con el supuesto de intersección aislada y llegadas aleatorias, pero impide modelar coordinación de corredor.
- **F9 [Bajo] — Demora de respaldo arbitraria.** Cuando la capacidad es 0, `d2` cae a la constante `300.0` s (`analysis.py:94`). Es un valor de relleno sin fundamento; convendría documentarlo o derivarlo.

### 3.3 Microsimulación — `simulator.py`

**Qué hace.** Simulación de paso discreto: por cada grupo de carriles mantiene una cola FIFO; en cada paso genera llegadas y, si la fase está en verde, descarga vehículos a ritmo de saturación. Reporta cola en el tiempo, espera promedio y servidos vs. llegados.

#### Hallazgos:

- **F11 [Alto] — Las llegadas no son Poisson.** El docstring afirma "Llegadas: Poisson", pero `simulator.py:110-114` hace: `n = int(mean); frac = mean - n; if random() < frac: n += 1`. Con los flujos típicos del proyecto la media por paso de 1 s es  $< 1$  (ej. 1150 veh/h  $\rightarrow$  0.32 veh/s), de modo que el proceso es en realidad **Bernoulli**: nunca puede generar 2 llegadas en un mismo paso. Para media  $> 1$ , la parte entera es determinista (varianza cero). El resultado es un proceso **sub-disperso**: la varianza de las llegadas es menor que la de un Poisson real. Como el sentido de una microsimulación estocástica es precisamente capturar las ráfagas que forman colas largas, esto **subestima sistemáticamente la cola máxima** y suaviza el resultado.
- **F12 [Alto] — Es una simulación de colas, no una microsimulación.** El modelo es de *cola vertical* (punto): no hay espacio, ni seguimiento vehicular (car-following), ni cambio de carril, ni conflictos de giro, ni bloqueo de carril compartido, ni *spillback* entre movimientos o accesos. Cada grupo de carriles es una cola FIFO independiente. Es un modelo legítimo y útil, pero llamarlo "microsimulación" lo equipara indebidamente con SUMO o Vissim, que resuelven dinámica continua en el espacio.
- **F13 [Medio] — Una sola corrida, una sola semilla.** `seed = 42` fijo (`models.py:184`). No hay réplicas múltiples, ni intervalos de confianza, ni descarte de periodo de calentamiento. Un resultado estocástico se presenta como estimación puntual; la práctica profesional corre N réplicas y reporta rangos.
- **F14 [Bajo] — Sin arranque perdido en la descarga.** Los vehículos descargan a saturación plena desde el segundo 0 del verde; en la realidad los primeros vehículos arrancan más lento.

### 3.4 Comparación de escenarios — `scenarios.py`

**Qué hace.** Aplica multiplicadores de demanda, reoptimiza y reanaliza cada escenario, elige el peor por demora y emite una estrategia recomendada.

#### Hallazgos:

- **F15 [Medio] — Escenarios solo con multiplicador uniforme.** Un escenario es un escalar aplicado a *toda* la demanda ( `scenarios.py:27-30` ). El crecimiento real es direccional y por movimiento (p. ej., un desarrollo nuevo carga un solo acceso). No se pueden modelar esos escenarios realistas.
- **F16 [Medio] — La "gestión de congestión" es una tabla de consulta.** La recomendación de estrategia ( `scenarios.py:34-63` ) es un `if` de cuatro ramas según la letra LOS que devuelve texto asesor. Es útil como guía, pero el plan de Copilot prometía "estrategias **adaptativas**": no hay control adaptativo ni optimización algorítmica de la gestión.

### 3.5 Análisis no semaforizado — `unsignalized.py`

**Qué hace.** TWSC (PARE en la secundaria) por aceptación de brechas con capacidad potencial, capacidad de movimiento e impedancia por rango; y glorieta con capacidad de entrada exponencial  $c = A \cdot e^{(-B \cdot vc)}$  .

**Fidelidad.** Es el módulo más cuidado: declara sus supuestos en el docstring (auditable), la fórmula de demora de control TWSC  $3600/c + 900T[...] + 5$  está correctamente transcrita y la estructura de impedancia por rango existe (algo que la mayoría de proyectos aficionados omite por completo).

#### Hallazgos:

- **F17 [Alto] — Coeficiente de glorieta desactualizado.** `unsignalized.py:283` usa  $A = 1130$  , que es el valor del HCM 2010. El HCM 6<sup>a</sup>/7<sup>a</sup> edición recalibró ese intercepto a **1380 veh/h** para glorieta de un carril. El proyecto, por tanto, **subestima la capacidad de entrada** respecto del estándar vigente.
- **F18 [Medio] — Capacidad de glorieta por multiplicación de carriles.** La capacidad se multiplica por el número de carriles de entrada ( `unsignalized.py:298` ). El HCM usa una ecuación por carril con coeficiente  $B$  distinto en cada uno; multiplicar la capacidad de un carril es una simplificación (sí está declarada en el docstring).
- **F19 [Medio] — Impedancia simplificada.** La impedancia es el producto directo de las probabilidades  $p_0$  ( `unsignalized.py:173-191` ). El HCM 2010 aplica además un factor de ajuste a la probabilidad de impedancia para el movimiento de rango 4 (giro izquierda menor); aquí se omite.
- **F20 [Bajo] — Cita de capítulo inconsistente.** El docstring atribuye la glorieta a "HCM 2010 cap. 22". En el HCM 2010 las glorietas son el **cap. 21**; el cap. 22 es el número de la 6<sup>a</sup> edición. La numeración mezcla ediciones. (TWSC sí es cap. 19 en el HCM 2010 — esa cita es correcta.)
- **F21 [Bajo] — Brechas críticas estáticas.** El diccionario `GAP` ( `unsignalized.py:40-45` ) no ajusta la brecha crítica por vehículos pesados, pendiente ni número de carriles, como sí hace el HCM. Tampoco hay aceptación de brechas en dos etapas (almacenamiento en mediana) — declarado.

### 3.6 Cuestiones transversales

- **F22 [Crítico] — Cero pruebas automatizadas.** No existe carpeta `tests/` ni un solo caso de prueba en el repositorio. Para un *motor de cálculo numérico* esto es la deuda más grave: cualquier refactor (corregir F2, F11, F17...) puede romper un resultado sin que nadie lo note. No hay red de seguridad.
- **F23 [Informativo] — Sin mecanismo de calibración.** No hay forma de ajustar el modelo contra aforos de campo. Esto es **deliberado y correcto** dado que el usuario no tiene datos; se anota para cuando los tenga (ver §8).

- **F24 [Bajo] — Persistencia inexistente.** El `.gitignore` reserva `data/runs/` pero nada escribe ahí; no se guardan corridas ni históricos.
- **F25 [Medio] — Sin modos no vehiculares.** El modelo de datos no contempla peatones, ciclistas ni transporte público en ninguna parte.

#### 4. Prometido vs. entregado (plan original de Copilot)

El asistente que originó el proyecto cerró su plan con siete módulos. Estado real, auditado:

| # | MÓDULO PROMETIDO                      | ENTREGADO                                                        | ESTADO                           |
|---|---------------------------------------|------------------------------------------------------------------|----------------------------------|
| 1 | Simulación (datos sintéticos)         | <code>simulator.py</code> — simulación de colas de paso discreto | Cumplido con reservas (F11, F12) |
| 2 | Cálculo de tiempos ("algoritmos")     | <code>optimizer.py</code> — un solo algoritmo (Webster)          | Parcial (F4)                     |
| 3 | Análisis de flujo (métricas)          | <code>analysis.py</code> — HCM 2010 cap. 18                      | Cumplido — supera lo pedido      |
| 4 | Simulación de escenarios              | <code>scenarios.py</code> — multiplicador uniforme               | Cumplido con reservas (F15)      |
| 5 | Gestión de congestión ("adaptativa")  | Tabla de consulta por LOS                                        | Parcial — no es adaptativa (F16) |
| 6 | Visualizaciones (gráficos y reportes) | <code>QueueChart</code> , 4 figuras, informe HTML→PDF            | Cumplido                         |
| 7 | Documentación completa                | README, instructivo de 18 secciones, informe                     | Cumplido — supera lo pedido      |

**Entregado de más, que Copilot no prometió:** análisis no semaforizado completo (TWSC + glorieta), mapa de ubicación geográfica, caso de ejemplo resuelto y reproducible, JSON importable, *pipeline* de PDF y de mapa regenerables, interfaz con tema *swiss*.

**Lectura honesta:** el proyecto entregó **más amplitud** de la prometida (módulo no semaforizado, geolocalización, documentación), pero con **menos profundidad** en dos promesas concretas — "algoritmos" de optimización (hay uno) y gestión "adaptativa" (es asesora, no adaptativa). El saldo es favorable, pero esas dos brechas están abiertas.

#### 5. Benchmark contra software profesional

##### 5.1 La brecha de estándar

| SOFTWARE             | NORMA DE CAPACIDAD          | ESTADO              |
|----------------------|-----------------------------|---------------------|
| <b>Este proyecto</b> | HCM 2010 (5ª ed.)           | Dos ediciones atrás |
| HCS 2025             | HCM 7 / 7.1 (seleccionable) | Vigente             |
| Synchro Studio 12    | HCM 7ª edición              | Vigente             |
| PTV Vistro           | HCM 6ª/7ª                   | Vigente             |

El *Highway Capacity Manual* va por su **7ª edición (HCM 2022)**, con la **actualización 7.1 de noviembre de 2025**. El proyecto cita la 5ª (2010).

## 5.2 Matriz funcional

Legenda: ● completo · ½ parcial / simplificado ○ ausente.

| CAPACIDAD                                 | ESTE PROYECTO    | HCS 2025 | SYNCHRO+SIMTRAFFIC | PTV VISSIM+VISTRO | SUMO      |
|-------------------------------------------|------------------|----------|--------------------|-------------------|-----------|
| Demora/capacidad por norma HCM            | ½ (2010)         | ●        | ●                  | ●                 | ½         |
| Optimización de tiempos                   | ½ (solo Webster) | ½        | ●                  | ●                 | ½         |
| Coordinación de corredor (onda verde)     | ○                | ○        | ●                  | ●                 | ●         |
| Control actuado / adaptativo              | ○                | ○        | ½                  | ●                 | ●         |
| Microsimulación (espacio + car-following) | ○                | ○        | ●                  | ●                 | ●         |
| Réplicas estocásticas + intervalos        | ○                | n/a      | ●                  | ●                 | ●         |
| TWSC / AWSC sin semáforo                  | ½ (solo TWSC)    | ●        | ●                  | ●                 | ½         |
| Glorietas                                 | ½ (2010)         | ●        | ●                  | ●                 | ½         |
| Peatones / ciclistas                      | ○                | ●        | ½                  | ●                 | ●         |
| Transporte público / prioridad            | ○                | ½        | ½                  | ●                 | ●         |
| Emisiones / combustible                   | ○                | ○        | ½                  | ●                 | ●         |
| Calibración con datos de campo            | ○                | ½        | ●                  | ●                 | ●         |
| Asignación dinámica en red                | ○                | ○        | ○                  | ●                 | ●         |
| API / automatización                      | ● (REST)         | ○        | ½                  | ●                 | ● (TraCI) |
| Costo de licencia                         | Gratis           | Pago     | Pago               | Pago (premium)    | Gratis    |
| Interfaz y docs en español                | ●                | ○        | ○                  | ½                 | ½         |

## 5.3 Detalle por herramienta

- **HCS 2025 (McTrans, U. Florida).** Implementación oficial del HCM; es la referencia normativa. Permite elegir entre metodología HCM 7 y 7.1. No optimiza ni microsimula: es la "calculadora oficial". *Nuestro paralelo:* el módulo `analysis.py` aspira a hacer lo que HCS hace, pero con la norma de 2010 y sin los factores de ajuste de saturación.
- **Synchro Studio 12 + SimTraffic (Cubic).** Estándar de facto para temporizado y coordinación de semáforos. Soporta HCM 7ª edición; cubre intersección semaforizada, AWSC, TWSC y glorieta; su fortaleza es la **optimización y coordinación de corredores** (reducir demoras, paradas, consumo y emisiones a lo largo de una arteria). SimTraffic le aporta la microsimulación. *Nuestra brecha mayor frente a Synchro:* no hay coordinación de corredor.
- **PTV Vissim + Vistro (PTV Group).** Vissim es microsimulación premium: modelo de seguimiento vehicular psico-físico de **Wiedemann** (Wiedemann 99, parámetros CC0–CC9), cambio de carril discreto, modelado de vehículos automatizados (ACC + cambio de carril automático). Vistro hace optimización de señales y estudios de impacto vial. Es el techo de fidelidad del mercado.
- **SUMO (Eclipse, código abierto).** Microsimulación microscópica y continua, multimodal (peatones, transporte público, ferrocarril), cálculo de emisiones (ruido y contaminantes), simulación de vehículos

autónomos y pelotones, y control remoto vía API **TraCI**. Última versión: enero de 2026. Es gratis, como nuestro proyecto, pero juega en otra liga de fidelidad. *Es el camino natural de integración* (ver §8).

- **Aimsun (Yunex/Siemens)**. Híbrido meso/microscópico con asignación dinámica de tráfico; fuerte para estudios de red. No se cubrió en detalle aquí; se menciona por completitud del panorama.

## 5.4 ¿Dónde encaja realmente nuestro proyecto?

No compite con Synchro ni Vissim, y está bien que no lo haga. Su nicho legítimo es el de **herramienta de tamizaje (screening) y docencia**: análisis preliminar rápido, gratuito, en español, sin instalación ni licencia, que funciona **sin datos históricos**. Ninguna herramienta profesional ocupa bien ese nicho — todas asumen aforos de campo y presupuesto. El error a evitar es que la interfaz o los informes hagan creer que el resultado tiene precisión de estudio formal.

## 6. Qué estamos haciendo bien

No todo es deuda. Conviene proteger estas fortalezas en cualquier refactor:

1. **Fundamento citable y mayormente correcto**. No hay fórmulas inventadas: Webster 1958, HCM 2010 cap. 18/19. `d1`, `d2`, la capacidad potencial por aceptación de brechas y la demora de control TWSC están bien transcritas. Un revisor puede auditar el cálculo contra el manual.
2. **Amplitud poco común**. Semaforizado, TWSC y glorieta en una sola herramienta liviana. En cobertura de *métodos de intersección* se acerca a Synchro; muy pocos proyectos abiertos lo logran.
3. **Funciona sin datos históricos**. Es una decisión de diseño deliberada y acertada para la situación real del usuario. Las herramientas profesionales prácticamente exigen aforos.
4. **Honestidad en los supuestos del módulo no semaforizado**. El docstring de `unsignalized.py` declara sus simplificaciones. Ese estándar de honestidad debería extenderse a los demás módulos (ver F6, F11).
5. **Robustez numérica**. Hay manejo de sobre-saturación, capado de `x`, topes de demora — el sistema no explota con NaN ni desbordes ante demanda extrema.
6. **Arquitectura limpia**. Modelos tipados con Pydantic, módulos separados por responsabilidad, API REST, *pipeline* de entregables reproducible (PDF, mapa, JSON importable, caso resuelto).
7. **Documentación muy por encima de lo habitual**. Instructivo de 18 secciones, informe de caso, todo en español.

## 7. Qué debemos mejorar — hallazgos priorizados

Cada ítem: problema, evidencia, impacto, recomendación y esfuerzo estimado.

### Crítico

**M1 · Crear una batería de pruebas automatizadas** (F22) - *Evidencia*: no existe `tests/` en el repositorio. - *Impacto*: sin red de seguridad, cualquier corrección de las de abajo puede introducir una regresión silenciosa en un número. - *Recomendación*: `pytest` con casos de regresión de valor conocido para Webster, `d1/d2`, TWSC y glorieta; idealmente un caso contrastado a mano contra el HCM. Hacerlo **antes** de tocar M3–M5. - *Esfuerzo*: medio. *Debe ir primero*.

### Alto

**M2 · Sincerar las etiquetas del modelo** (F6, F11, F12) - *Problema*: el código afirma "Poisson" y "cola 95-percentil" para cosas que no lo son, y llama "microsimulación" a una simulación de colas. - *Recomendación*, en



*orden de menor a mayor esfuerzo:* (a) corregir las etiquetas y docstrings para que digan lo que el modelo hace (Bernoulli/colas) — esfuerzo bajo, **gana credibilidad de inmediato**; (b) implementar un muestreo Poisson real ( `random.expovariate` o conteo Poisson) — esfuerzo bajo; (c) reemplazar `q_95 = 2 * q_avg` por el modelo de cola del HCM — esfuerzo medio. - *Esfuerzo:* bajo a medio.

**M3 · Actualizar a HCM 7ª edición** (F10, F17) - *Problema:* norma dos ediciones atrasada; coeficiente de glorieta `1130` ya recalibrado a `1380`. - *Recomendación:* migrar coeficientes y factores de ajuste de saturación del HCM 7; empezar por la glorieta, que es un cambio puntual y de alto efecto. - *Esfuerzo:* medio (alto si se hace toda la cadena de factores de saturación).

**M4 · Segundo algoritmo de optimización** (F1, F4) - *Problema:* Webster sobrestima el ciclo en el régimen congestionado que es, precisamente, el caso del usuario. - *Recomendación:* añadir una optimización por **minimización directa de la demora HCM** (búsqueda del ciclo que minimiza `d` agregada). Ofrecer ambos resultados y dejar que el usuario compare. - *Esfuerzo:* medio.

**M5 · Coordinación de corredor** (brecha vs. Synchro) - *Problema:* cada intersección se trata como aislada; es la ausencia más visible frente al software profesional. - *Recomendación:* modelar *offsets* entre intersecciones vecinas y un factor de progresión `PF` real (hoy fijo en 1.0). Es un salto de alcance grande. - *Esfuerzo:* alto.

## Medio

- **M6 · Réplicas estocásticas** (F13): correr N simulaciones con semillas distintas y reportar rango/percentiles en vez de un punto. *Esfuerzo:* bajo.
- **M7 · Corregir el bug de re-acotamiento del verde** (F2): volver a acotar tras el reescalado en `optimizer.py`. *Esfuerzo:* bajo.
- **M8 · Escenarios direccionales** (F15): permitir multiplicadores por acceso o por movimiento, no solo uniformes. *Esfuerzo:* bajo.
- **M9 · Cadena de factores de saturación HCM** (F8): exponer ancho de carril, pendiente, pesados y peatones como entradas. *Esfuerzo:* medio.
- **M10 · Modos peatonal y ciclista** (F25): el HCM 7 los trata como modos de primera clase. *Esfuerzo:* alto.

## Bajo

- **M11 · Gancho de calibración** (F23): cuando el usuario consiga aforos, permitir ajustar flujo de saturación y brechas críticas contra ellos.
- **M12 · Limpieza** (F3, F5, F9, F20): aritmética muerta del tiempo perdido, distinción verde efectivo/visualizado, demora de respaldo, cita de capítulo.
- **M13 · Persistencia de corridas** (F24): guardar y comparar análisis en `data/runs/`.

## 8. Hoja de ruta sugerida

Tres fases. La primera es condición para las demás.

**Fase 0 — Cimientos (antes de cualquier otra cosa).** M1 (pruebas) + M2a (sincerar etiquetas) + M7 (bug de verde). Con esto el proyecto queda *honesto* y *protegido contra regresiones*. Esfuerzo bajo-medio, máximo retorno.

**Fase 1 — Rigor del cálculo.** M2b/M2c (Poisson real + cola HCM) + M3 (HCM 7 y glorieta 1380) + M4 (segundo optimizador) + M6 (réplicas) + M8 (escenarios direccionales) + M9 (factores de saturación). Al terminar, el motor es defendible frente a un revisor técnico.



**Fase 2 — Alcance de red y multimodal.** M5 (coordinación de corredor) + M10 (peatones/ciclistas) + control actuado/ adaptativo. Aquí conviene **no reinventar SUMO**: la vía más rentable es *integrar* SUMO como motor de microsimulación de alta fidelidad vía su API TraCI, y conservar nuestro backend como capa de análisis HCM, optimización y reporte en español. Eso convierte la mayor debilidad (F12) en una fortaleza sin reescribir un simulador desde cero.

Estas fases son coherentes con la sección "Próximos pasos" del `README.md`, pero la priorizan: **primero honestidad y pruebas, después rigor, después alcance.**

---

## 9. Conclusión

---

El proyecto hace bien lo esencial: aplica teoría reconocida, cubre más tipos de intersección de los prometidos, funciona sin datos y está excepcionalmente bien documentado en español. Su problema no es la corrección de las fórmulas — están, en su mayoría, bien transcritas — sino **tres cosas concretas y arreglables**: (1) afirma ser más de lo que es en tres etiquetas clave (Poisson, cola 95-percentil, microsimulación); (2) se apoya en una edición del HCM atrasada y en un Webster que flaquea justo en el régimen congestionado del caso real; y (3) no tiene una sola prueba que lo proteja.

Ninguna de las tres es difícil de corregir. La recomendación central de esta investigación es simple y de bajo costo: **empezar por la Fase 0** — escribir pruebas y sincerar las etiquetas. Eso transforma el proyecto de "demo prometidora" en "herramienta de tamizaje confiable y honesta" sin escribir casi código nuevo. El rigor y el alcance vienen después, sobre esa base.

---

## Fuentes

---

- Highway Capacity Manual — Wikipedia
- Highway Capacity Manual 7th Edition — National Academies Press
- HCM Edition 7.1 (noviembre 2025) — National Academies Press
- HCS 2025 — McTrans Center, University of Florida
- HCM 6th Edition: Roundabout Calculation Changes — MikeOnTraffic
- An Assessment of the HCM Edition 6 Roundabout Capacity Model — ResearchGate
- Eclipse SUMO — Simulation of Urban MObility
- Eclipse SUMO — repositorio oficial (GitHub)
- Synchro Studio 12 — notas de versión (Cubic)
- Cubic actualiza Synchro Studio (soporte HCM 7)
- PTV Vissim — Wikipedia
- Computing optimum traffic signal cycle length — ScienceDirect
- Traffic Light Optimization Based on Modified Webster Function — Wiley